GitHub Actions & Backend Deployment

A Clear, Step-by-Step Explanation for Beginners

While your infrastructure workflow manages the foundational cloud environment, this workflow is responsible for shipping the actual application code into that system. It packages your backend into a Docker container, uploads it to AWS, and replaces old active instances with fresh code cleanly without causing down-time.

1. Triggers & Concurrency Guardrails

Workflow Name & Triggers

```
name: Backend image

on:
  push:
    branches: [main]
    paths: ["app/backend/**", ".github/workflows/backend.yml"]
```

- **Explanation:** This automation robot only wakes up when code changes are merged or pushed directly into the `main` branch, and only if changes happen inside your `app/backend/` directory or to this workflow YAML file.

Concurrency Queue Control

```
concurrency:
  group: backend-${{ github.ref }}
  cancel-in-progress: false
```

- **Explanation:** AWS Auto Scaling Groups (ASG) strictly allow only one rolling instance software deployment/refresh at any single moment. If two parallel runs tried to update the servers simultaneously, they would crash into each other. Setting `cancel-in-progress: false` forces the secondary deployment to queue up cleanly instead of colliding.

2. Step-by-Step Execution Sequence

GitHub starts up a clean, isolated **Ubuntu Linux** cloud runner server environment and walks through your instructions one by one:

Step 1 & 2: Fetch Code & AWS Login

```
- uses: actions/checkout@v4
- uses: aws-actions/configure-aws-credentials@v4
```

Action: Clones your code onto the empty server runner and leverages secure OIDC to authenticate with AWS temporarily without static, risky passwords.

Step 3: Authenticate Container Registry

```
- uses: aws-actions/amazon-ecr-login@v2
```

Action: Logs the runner into your AWS Elastic Container Registry (ECR). This securely authorizes the runner's Docker client engine so it has immediate clearance to push custom images into your cloud system storage.

Step 4: Read Infrastructure Targets

```
- name: Read ECR URL and ASG name from compute state
  run: |
    cd terraform/compute
    terraform init -input=false
    {
      echo "REPO=$(terraform output -raw ecr_repository_url)"
      echo "ASG=$(terraform output -raw asg_name)"
    } >> "$GITHUB_ENV"
```

Action: The runner doesn't have your specific resource names hardcoded. It temporarily runs inside your `terraform/compute` directory to parse the exact live backend outputs. It queries your unique ECR repository address and Auto Scaling Group identifier, setting them as temporary environment variables (`$REPO` and `$ASG`).

Step 5: Package & Build Docker Container

```
- name: Build & push image
  run: |
    cd app/backend
    docker build -t "$REPO:${GITHUB_SHA}" -t "$REPO:latest" .
    docker push "$REPO:${GITHUB_SHA}"
    docker push "$REPO:latest"
```

Action: Enters your application directory, executes a local container image compilation, and sets two labels: an absolute target string tracking your specific Git commit (`${GITHUB_SHA}`), and a floating pointer label (`:latest`). It uploads both elements to your remote cloud registry.

Step 6: Zero-Downtime Rolling Update

```
REFRESH_ID=$(aws autoscaling start-instance-refresh   --auto-scaling-group-name
"$ASG"   --preferences MinHealthyPercentage=50 ...)
```

Action: Triggers a safe rolling replacement of live infrastructure instances. It targets a threshold parameter keeping at least 50% of application servers active and healthy while updating. The `while true` shell script blocks and polls AWS status metrics every 30 seconds until the rolling update concludes successfully or halts on errors.

> ### Summary of Your Complete CI/CD Ecosystem
>
> - **Infrastructure Tier (terraform.yml):** Configures and stabilizes cloud networks, security group boundaries, container registries, and scaling parameters.
> - **Application Tier (backend.yml):** Packages your core application software, tracks history via commits, and updates live servers systematically with zero application down-time.